Comparing Neural Networks and Linear Regression for Schnapsen Agents

Anonymous Submission

Vrije Universiteit (VU), De Boelelaan 1105, 1081 HV Amsterdam

Abstract. The game of Schnapsen provides a challenging test environment for machine learning algorithms, offering a combination of imperfect and perfect information phases. In this article, we look at the suitability of neural networks (NN) and linear regression (LR) in developing a competitive Schnapsen game-playing agent. We compare the algorithms with reference to total wins, how they perform through different phases of the game, and the efficiency of their learning. By leveraging a dataset generated through self-play between bots, we explore how these models adapt to the complex decision-making required in Schnapsen. The results identify the strengths and limitations of each model and provide useful information on the characteristics of a successful machine learning solution to games of imperfect information. The research adds to overall knowledge of machine learning in strategic environments.

Keywords: Machine Learning · Neural Networks · Linear Regression · Schnapsen · Imperfect Information.

1 Introduction

Albert Einstein's famous quote, "Games are the most elevated form of investigation," highlights the educational and strategic potential of games like Schnapsen. These games foster dynamic decision-making and problem-solving skills (von Gillern, 2018). The application of machine learning (ML) in games, as exemplified by AlphaGo's victory over Lee Sedol in Go, illustrates the rapid advancements in AI research (Chang et al., 2016). While Go and Chess often dominate the spotlight, Schnapsen, with its distinct imperfect and perfect information phases, presents a unique opportunity for evaluating ML models (Kouwenhoven et al., 2018).

Played with a deck of 20 cards, aces, tens, kings, queens, and jacks, the game asks players to obtain 66 points by acquiring bribes or proclaiming a marriage (king and queen of trump). The game tests individuals for the ability to balance immediate strategic actions with longer-range strategic plans, particularly as players transition from doubt to complete understanding. These characteristics require a machine learning model capable of dynamic reasoning and adaptive decision-making.

Over the past decade, machine learning (ML) technology has revolutionized the world of strategy games. Some of the milestones include AlphaGo, which combined neural networks with Monte Carlo tree search methods to achieve superhuman performance in playing Go. Likewise, uses of ML for imperfect information games, such as poker, have demonstrated the capabilities of algorithms to play well under uncertainty. Schnapsen, with its small state space and biphasic nature, presents a fascinating setting in which to study the performance of basic models, i.e., neural networks (NNs) and linear regression (LR), under such dynamic conditions.

Games like Schnapsen require the strategic coordination of probability, decision-making, and adaptability, making them more than just a challenge for human players but also a demanding testbed for artificial intelligence. Unlike other card games, Schnapsen's structured progression from imperfect to perfect information adds a layer of complexity rarely found in similar environments. The combination of sequential reasoning, hidden variables, and rapidly changing decision states makes it an ideal scenario for studying ML model performance. Furthermore, the game offers opportunities to observe how well algorithms generalize across different game scenarios, emphasizing not only predictive accuracy but also adaptability to novel situations.

This study investigates the research question: ***"Which machine learning models work best, and what makes a good machine learning model for Schnapsen?"*** *Neural networks are renowned for their ability to capture non-linear patterns and adapt to different game states, making them suitable for the perfect information phase. Linear regression, on the other hand, is characterized by its simplicity, computational efficiency, and interpretability, serving as a valuable basis for comparison. By systematically evaluating these models on parameters such as number of wins, phase-dependent performance, and learning efficiency, this study seeks to identify their respective advantages and disadvantages. Designing an efficient agent for the Schnapsen game requires a balance between computational requirements and strategic adaptability. This study evaluates how well NN and LR perform in two-phase Schnapsen problems, which contributes to the model selection for games with imperfect information. The rest of this report is organized as follows: First, we provide detailed background information about Schnapsen and the ML models used. Next, we describe the experimental setup, including the methodology for data generation and model training. Finally, we present and analyze the results, discuss their implications, and conclude with recommendations for future research.*

2 Background Information

2.1 The Game 'Schnapsen'

Schnapsen, a traditional Austrian card game, is played with a 20-card deck and requires players to accumulate 66 points by winning tricks or declaring marriages

(a king and queen in the trump suit). Its rules and gameplay structure have been well-documented (Schnapsen and Sixty-Six Rules, 2018). The dual-phase nature of the game — imperfect information in Phase 1 and perfect information in Phase 2 — adds complexity and makes it an ideal testbed for AI techniques (Kouwenhoven et al., 2018).

In the imperfect information phase, stocks remain hidden and players have to make decisions based on limited observable data. Probabilistic reasoning, opponent modeling, and risk management play a crucial role here, as players must evaluate potential outcomes without complete information. When the supply is exhausted or closed, the game enters the perfect information phase, where all remaining cards are revealed. Players must then implement precise strategies to optimize their chances of winning. This transition between phases, requiring both probabilistic inference and deterministic planning, makes Schnapsen a unique area for studying machine learning models.

2.2 Challenges in Machine Learning for Schnapsen

Modeling imperfect information games like Schnapsen presents significant challenges, requiring ML models to adapt dynamically across complex game states. In the imperfect information phase, models must infer hidden states and predict the opponent's hand, which introduces uncertainty. Simpler models like linear regression struggle in this phase, while neural networks, with their ability to model intricate feature interactions, handle ambiguity more effectively (Silver et al., 2020; Salzer, 2023).

The perfect information phase simplifies decision-making by revealing the complete game state, allowing both models to optimize strategies. However, the transition between these phases tests a model's adaptability, as strategies must adjust to the changing nature of available information. Schnapsen's dual-phase structure thus serves as an ideal benchmark for evaluating the robustness and efficiency of ML models.

2.3 Machine Learning Framework

To address these challenges, this study utilizes Neural Networks (NNs) and Linear Regression (LR) as representative models from opposite ends of the complexity spectrum. NNs excel at capturing intricate, non-linear relationships in game data, making them suitable for dynamic and complex decision-making scenarios. On the other hand, LR offers simplicity, computational efficiency, and faster convergence, making it a reliable baseline for structured environments (Ashmore et al., 2021). This dual approach enables an exploration of trade-offs between complexity, adaptability, and computational demands.

2.4 Neural Networks

Neural networks (NNs) are powerful tools for capturing nonlinear relationships and adapting to dynamic decision spaces. Their architecture, consisting of interconnected neurons organized in layers, allows us to model complex interactions between features. Schnapsen NNs are excellent at using information such as card distribution, player scores, and stock status to make strategic decisions.

The NN model used in this study is a multilayer perceptron (MLP) with a single hidden layer of 30 neurons. This configuration strikes a balance between complexity and computational efficiency, allowing the model to generalize to different game scenarios. Although NNs require larger datasets and more computational resources than simpler models, their flexibility makes them particularly effective for the perfect information phase, when all game states are fully observable.

2.5 Linear Regression

Linear regression (LR) is a simple and straightforward machine learning algorithm that models the relationship between input features and output variable by fitting a linear equation to the data. Its computational efficiency and stability make it a valuable basic framework for basic research, especially in structured and less dynamic scenarios. However, linear regression is limited by its inability to model nonlinear feature interactions, making it less effective for the Schnapsen imperfect information phase. In this phase, where uncertainty and incomplete data prevail, LR cannot deduce hidden states and adapt to dynamic changes. Nevertheless, its simplicity allows fast training and evaluation, making it a practical choice for the perfect information phase, where the relationships between features are more simple and well-defined.

The dataset for this study was created by simulating games between bots, which took into account characteristics such as player points, cards in hand, trump suits, and phases of play. These characteristics were used to train and evaluate both models, focusing on win rates, phase-dependent performance, and learning efficiency. By comparing the strengths and weaknesses of neural networks and linear regression, this study aims to provide a framework for understanding effective machine learning strategies in Schnapsen.

3 Research Question and Hypotheses

The complexity of Schnapsen, with its dual-phase structure, raises fundamental questions about the effectiveness of different machine learning models in strategic gameplay. This research investigates the following core question:

Which machine learning models work best, and what makes a good machine learning model for Schnapsen?

To address this, the study evaluates the performance of Neural Networks and Linear Regression across several metrics, including win rates, phase-specific effectiveness, and training efficiency. The following hypotheses guide the research:

3.1 Hypotheses

Hypothesis 1:

Neural Networks (NNs) will outperform Linear Regression (LR) in overall win rates and phase- specific performance, particularly in the perfect information phase (Phase 2). This is due to NNs' ability to model non-linear relationships and adapt to complex, dynamic strategies observed in late- game scenarios.

Hypothesis 2: Linear Regression will demonstrate greater stability in results and faster convergence during training, particularly when working with smaller datasets or in scenarios where computational resources are constrained. However, its performance is expected to plateau earlier than NNs as the training size increases.

These hypotheses are grounded in the inherent differences between the models. Neural Networks are better suited for dynamic, complex decision-making scenarios, while Linear Regression offers simplicity and efficiency as a baseline model. The results of this study are expected to provide insights into the design and application of machine learning models for strategic games, particularly those involving imperfect information.

4 Experimental Setup

This section outlines the experimental design and methodology employed to evaluate the performance of Neural Networks (NNs) and Linear Regression (LR) models in Schnapsen. The experimental framework draws inspiration from studies on games with imperfect information, such as Monte Carlo Tree Search in Go (Silver et al., 2020) and Perfect Information Monte Carlo Sampling (PIMCS) rollout strategies (Kouwenhoven et al., 2018). By simulating games between bots, the dataset captures a wide range of game scenarios, allowing comprehensive evaluation. The Phase2FocusedBot was designed with strategic depth to test the adaptability of ML models, a technique supported by prior research on reinforcement learning for games (Salzer, 2023). Below, we detail the key components of the experimental setup.

4.1 Data Generation

The dataset was generated through extensive simulations of Schnapsen games involving bots with varying strategies. Each game produced data points that captured essential game states, including player scores, cards in hand, trump suits,

and the phase of the game (imperfect or perfect information). These features provided a rich set of inputs for training and evaluating machine learning models.

To explore the impact of training size, datasets were created with 2,000, 5,000, 8,000, 10,000, 15,000, and 20,000 games. The datasets were balanced to ensure an equal representation of win/loss scenarios, enhancing the models' ability to generalize. Each dataset contained approximately 56,233 samples per class for the 8,000-game configuration, with similar proportions maintained across other sizes.

4.2 Training Configuration

The experiments evaluated two models:

Neural Networks (NNs):

The NN model employed a Multi-Layer Perceptron (MLP) architecture with a single hidden layer of 30 neurons, utilizing the tanh activation function and early stopping to prevent overfitting. Training times for the NN varied based on the dataset size, starting at 0.081 minutes for 2,000 games, 0.907 minutes for 10,000 games, and reaching 1.478 minutes for 20,000 games. On average, the NN required approximately 0.73 minutes across all training sizes. This model's ability to capture non-linear relationships made it particularly well-suited for modeling complex game dynamics, excelling in the perfect information phase where precise decision-making is critical. The balance between computational cost and performance scalability underscores its effectiveness in strategic environments.

Linear Regression (LR):

The LR model served as a computationally efficient baseline. It required significantly less training time, averaging 0.34 minutes for the same dataset size. While limited to modeling linear relationships, its simplicity and interpretability provided valuable insights into foundational strategies in Schnapsen.

4.3 Baseline Opponents

The models' performance was tested against five distinct bots, each designed to represent varying levels of strategic complexity:

1. **Random Bot**:Selects moves randomly, providing a baseline comparison.
2. **Bully Bot**: Prioritizes high-value cards without considering long-term strategy.
3. **Phase2FocusedBot**: A sophisticated agent optimized for Phase 2 decision-making. It employs a detailed Move Management system to handle marriages, trump exchanges, and card evaluation based on game state.

i. **Phase 1 Strategy**: Focuses on prioritizing trump marriages and managing trump exchanges, while defaulting to random moves when no special plays are available.

ii. **Phase 2 Strategy**: Differentiates between leader and follower positions, tracking high-value cards, optimizing trump usage, and minimizing point loss risks. Continuous position evaluation tracks metrics such as trump card count, high-value card count, and hand potential.

4. **RDeep-Basic**: Implements basic lookahead strategies, providing a moderate challenge.

5. **RDeep-Advanced**: Utilizes advanced heuristics and decision trees, representing the highest level of competition.

4.4 Evaluation Metrics

To comprehensively assess model performance, the following metrics were defined:

1. **Win Rates**: The percentage of games won against each opponent, serving as the primary performance indicator.
2. **Phase-Specific Success**: A breakdown of win rates in Phase 1 (imperfect information) and Phase 2 (perfect information). This metric highlights each model's ability to adapt to different game phases.
3. **Training Efficiency**: Measured in terms of training time and the number of iterations required for convergence.

4.5 Implementation Details

The experiments were implemented in Python, leveraging a robust and modular codebase. The Phase2FocusedBot, a key component, was designed with a clean separation of concerns to ensure maintainability and scalability. Modular strategy components allowed for precise control over different game phases, while type-safe constructs and error handling minimized runtime issues.

The simulations followed a position-balanced evaluation framework, playing 200 games per matchup to ensure consistency across different configurations. The results were compiled into a comprehensive dataset, enabling detailed analyses of model performance against each opponent and across training sizes.

This setup provided a controlled, repeatable framework for evaluating the effectiveness of NNs and LR in the strategic environment of Schnapsen. By incorporating diverse opponents and extensive datasets, the experiments offered a robust foundation for comparing the strengths and limitations of the models in dynamic, imperfect information games.

5 Results

This section presents the performance evaluation of Neural Networks (NNs) and Linear Regression (LR) models in Schnapsen, focusing on win rates, phase-specific success, and training efficiency. Results are structured around varying training sizes and opponent strategies, emphasizing key trends and observations.

5.1 Neural Network Performance

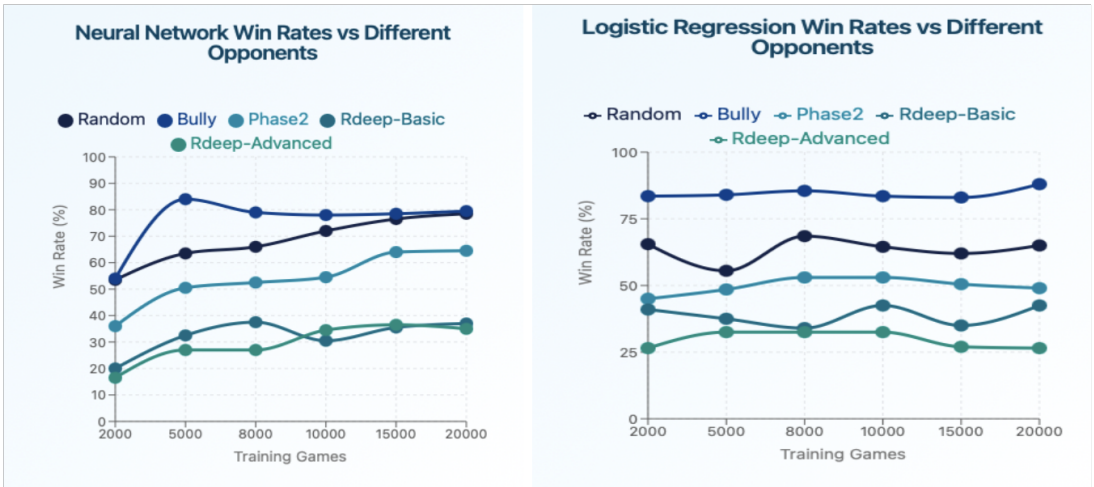
NNs exhibited steady improvement across training sizes, peaking at a 58.9% win rate for 20,000 games. The model demonstrated exceptional adaptability in dynamic game states, particularly in Phase 2, where it consistently outperformed LR. This aligns with prior findings highlighting the potential of deep learning in leveraging perfect information during gameplay (Silver et al., 2020). Key results include:

- **Random Bot**: The NN achieved its best performance against this baseline opponent, with a win rate peaking at 78.5% at 20,000 games. Phase 2 played a critical role, contributing 57.5% of the victories.
- **Bully Bot**: Performance reached 79.5% at 20,000 games, highlighting the NN's capability to counter simplistic, high-value prioritization strategies effectively.
- **Phase2FocusedBot**: The NN improved significantly, with win rates increasing from 36.0% at 2,000 games to 64.5% at 20,000 games. Phase 2 dominance was evident, reflecting the model's strategic depth.
- **RDeep Variants**: Against RDeep-Basic and RDeep-Advanced bots, NN performance plateaued at 37.0% and 35.0%, respectively. The advanced heuristic strategies posed a consistent challenge.

5.2 Logistic Regression Performance

LR excelled in Phase 1 scenarios, with peak performance observed at 10,000 games (54.2% win rate). This performance reflects its computational efficiency and suitability for simpler decision-making tasks, as suggested in foundational ML research (Ashmore et al., 2021). However, its plateau in performance beyond 10,000 games highlights its limitations in adapting to dynamic and non-linear interactions.Key observations include:

- **Random Bot**: The LR model maintained stable win rates, averaging 65% across all training sizes. Its peak performance was 66.5% at 10,000 games, with Phase 2 contributing significantly.
- **Bully Bot**: Excelling against this opponent, LR consistently achieved win rates above 83%, peaking at 88% at 20,000 games. This highlights its effectiveness in simpler, predictable scenarios.
- **Phase2FocusedBot**: LR showed mixed results, with win rates varying between 45% and 53% across training sizes. It struggled to consistently adapt to strategic similarities.
- **RDeep Variants**: LR performed better against RDeep-Basic (up to 42.5%) but faced significant challenges with RDeep-Advanced (26.5%).



5.3 Comparison of Models

Training Efficiency

Linear Regression demonstrated higher training efficiency, reaching peak performance faster and with less computational cost. For the 2,000-game dataset, LR achieved a 52.3% win rate in just 0.076 minutes, while NN achieved 36.0% in 0.081 minutes. LR peaked at 54.0% with 10,000 games but plateaued afterward. NN, although slower to converge, steadily improved, reaching a 58.9% win rate with 20,000 games, requiring 1.478 minutes of training on average.

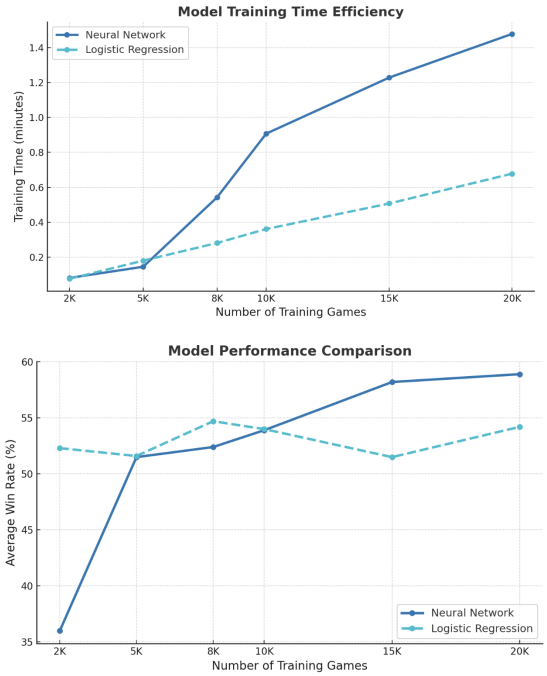
Phase-Specific Performance

Both models dominated in Phase 2 due to the availability of perfect information. NN consistently outperformed LR in Phase 2, achieving 57.0% against Phase2FocusedBot. In Phase 1, LR showed better stability and efficiency in simpler decision-making scenarios, outperforming NN with smaller training datasets.

Opponent Performance

Neural Networks performed well against dynamic and unpredictable opponents, achieving a 78.5% win rate against the Random Bot and improving to 64.5% against the Phase2FocusedBot after 20,000 games. However, NN struggled with advanced opponents like RDeep-Advanced, achieving only 35.0%.

Conversely, LR excelled in simpler scenarios, peaking at 88.0% against the Bully Bot and maintaining 65.0% against the Random Bot. Against more complex opponents like RDeep-Advanced, LR struggled, with a win rate of just 26.5%.



6 Findings and Discussion

The experimental results offer critical insights into the comparative performance of Neural Networks (NNs) and Linear Regression (LR) in Schnapsen. Neural Networks demonstrated adaptability by improving against dynamic strategies like Phase2FocusedBot (36% → 64.5%) but struggled with advanced heuristic strategies employed by RDeep variants (Salzer, 2023). Their performance showed steady improvement with larger datasets, particularly in Phase 2, where perfect information enabled more effective decision-making (Silver et al., 2020). Conversely, LR excelled against simpler opponents, achieving peak win rates of 88% against Bully, though its performance against Phase2FocusedBot fluctuated (45–53%) and plateaued after 10,000 games (Ashmore et al., 2021).

General trends indicate both models dominated Phase 2, leveraging the advantages of perfect information, while Phase 1 remained challenging due to its inherent uncertainty (Cameron & Thore, 2018). NN required larger training datasets to reach optimal performance, whereas LR performed well with smaller datasets but offered diminishing returns with extended training (Chang et al., 2016).

These findings highlight key trade-offs between complexity and efficiency. Neural Networks are suited for tasks demanding dynamic adaptability and strategic depth but require greater computational resources. Linear Regression, on the

other hand, is computationally efficient and excels in simpler scenarios, making it ideal for resource-constrained environments (Ashmore et al., 2021). However, its limitations in handling complex strategies underscore the need for hybrid approaches or reinforcement learning to address challenges posed by advanced opponents like RDeep-Advanced (Salzer, 2023; Silver et al., 2020).

7 Future Work

Future work should focus on developing hybrid models that combine the adaptability of Neural Networks (NNs) with the computational efficiency of Linear Regression (LR). LR could be employed for Phase 1 decision-making, where data is sparse and patterns are simpler, while NNs could handle the complex interactions in Phase 2 under perfect information. Such a system could dynamically switch between models, leveraging the strengths of both approaches to improve efficiency and adaptability (Ashmore et al., 2021; Silver et al., 2020).

Expanding the dataset to include diverse and adversarial scenarios is another key avenue for future research. Incorporating multi-agent reinforcement learning could challenge the adaptability of hybrid models and validate their performance in dynamic environments (Salzer, 2023). Additionally, optimizing training efficiency through transfer learning or incremental training techniques could reduce computational demands while preserving NNs' strategic reasoning capabilities.

Finally, advancing evaluation metrics beyond win rates to assess decision quality and adaptability to specific game states could offer deeper insights into model performance. These refinements could generalize findings to other dual-phase games, extending the applicability of hybrid frameworks to a broader range of imperfect information games (Silver et al., 2020; Ashmore et al., 2021).

8 Conclusions

This study evaluated the performance of Neural Networks (NNs) and Linear Regression (LR) models in Schnapsen, a two-player card game characterized by its dual-phase structure of imperfect and perfect information. The research explored the question: "Which machine learning models work best, and what makes a good machine learning model for Schnapsen?" Through systematic comparisons across training sizes and opponent types, the study highlighted the trade-offs between adaptability, computational efficiency, and scalability.

Neural Networks demonstrated superior adaptability and dynamic decision-making, particularly in Phase 2, where perfect information allowed them to model non-linear relationships effectively. Their win rate peaked at 78.5% against the Random Bot after 20,000 training games. However, their reliance on larger datasets and substantial computational resources limits their scalability (Silver et al., 2020; Chang et al., 2016).

Linear Regression emerged as a computationally efficient baseline, excelling in simpler scenarios such as games against the Bully Bot, where it achieved a peak win rate of 88.0%. Its faster convergence made it suitable for resource-constrained environments. However, LR struggled with more complex strategies, plateauing after 10,000 games due to its inability to capture non-linear interactions (Ashmore et al., 2021).

Both models performed well in Phase 2, where complete state information enables optimal strategies, but their consistent underperformance in Phase 1 highlights the challenge of modeling uncertainty. Neither model sufficiently addressed probabilistic reasoning, leaving Phase 1 as an area requiring further exploration (Salzer, 2023).

This study emphasizes the trade-offs between model complexity and computational efficiency. While NNs excel in strategic adaptability, LR provides a practical solution for simpler environments. These findings align with broader trends in AI, such as AlphaGo's integration of deep learning with heuristic approaches (Silver et al., 2020).

Future research should prioritize improving Phase 1 performance through reinforcement learning or hybrid approaches that leverage the strengths of both models. Expanding the dataset and testing other game environments with similar dual-phase structures could further generalize these findings. Additionally, ensemble models and multi-agent systems could enhance adaptability and computational efficiency, advancing AI solutions for imperfect information games.

By identifying these strengths and limitations, this research contributes to the understanding of machine learning applications in strategic games and provides a foundation for designing adaptable and efficient ML agents.

9 References:

References

1. von Gillern, S.: Games and Their Embodied Learning Principles in the Classroom: Connecting Learning Theory to Practice. IGI Global (2018). https://doi.org/10.4018/978-1-5225-5198-0.ch029
2. Chang, H. S., Fu, M. C., Hu, J., Marcus, S. I.: Google DeepMind's AlphaGo: operations research's unheralded role in the path-breaking achievement. OR/MS Today **43**(5), 24+ (2016). Retrieved from https://link.gale.com
3. Schnapsen and Sixty-Six Rules. Accessed 31 January 2018. http://www.psellos.com
4. Kouwenhoven, S., Maaiveld, T., Metekia, M.: Comparing Naive Bayes and Logistic Regression as Learning Models for PIMCS Rollout Policy in the Card Game Schnapsen. ISVU (2018).
5. Salzer, T.: Reinforcement Learning for Games with Imperfect Information. Springer (2023).

6. Ashmore, R., Calinescu, R., Paterson, C.: Assuring the Machine Learning Lifecycle: Desiderata, Methods, and Challenges. ACM Computing Surveys **54**(5), Article 111 (2021). https://doi.org/10.1145/3453444
7. Silver, D., Schrittwieser, J., Simonyan, K., et al.: Combining Deep Reinforcement Learning and Search for Imperfect Information Games. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1–12 (2020).
8. Cameron, M., Thore, M.: Machine Learning and Games: Impacts on Decision-Making Strategies. Elsevier (2018).

10 Appendix:

```python
from schnapsen.game import Bot, PlayerPerspective, Move, SchnapsenGamePlayEngine, Score, GamePhase
from schnapsen.deck import Card, Rank, Suit
from schnapsen.bots import RandBot, BullyBot, RdeepBot
from schnapsen.bots.ml_bot import MLDataBot, MLPlayingBot, MLPClassifier, train_ML_model
import pathlib
import random
import time
import shutil
from typing import Optional, cast, List, Dict, Tuple
import traceback
from schnapsen.game import SchnapsenTrickScorer
import os

class Phase2FocusedBot(Bot):
    """Bot focused on Phase 2 excellence while maintaining simple Phase 1 strategy."""

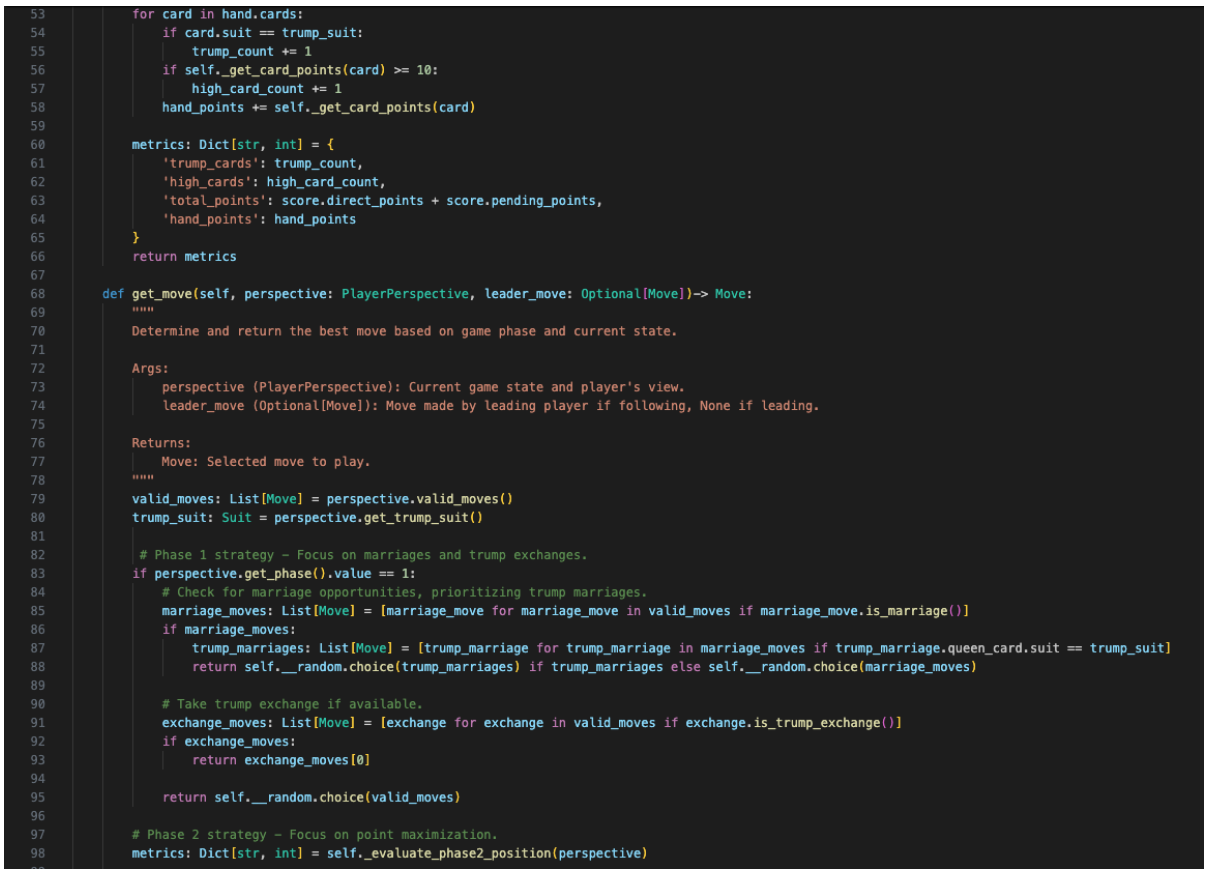
    def __init__(self, rand: random.Random)-> None:
        super().__init__()
        self.__random: random.Random = rand

    def _get_move_card(self, move: Move)-> Optional[Card]:
        """Extract card from any move type safely - returns queen for marriages and jack for exchanges."""
        if move.is_marriage():
            return move.queen_card
        elif move.is_trump_exchange():
            return move.jack
        elif hasattr(move, 'card'):
            return move.card
        return None

    def _get_card_points(self, card: Card)-> int:
        """Calculate card points according to Schnapsen scoring rules."""
        points: Dict[Rank, int] = {
            Rank.ACE: 11,
            Rank.TEN: 10,
            Rank.KING: 4,
            Rank.QUEEN: 3,
            Rank.JACK: 2
        }
        return points.get(card.rank, 0)

    def _evaluate_phase2_position(self, perspective: PlayerPerspective)-> Dict[str, int]:
        """Analyze current game state for Phase 2 strategy decisions."""
        hand = perspective.get_hand()
        trump_suit = perspective.get_trump_suit()
        score = perspective.get_my_score()

        # Count trump and high-value cards in hand.
        trump_count: int = 0
        high_card_count: int = 0
        hand_points: int = 0
```



```
        for card in hand.cards:
            if card.suit == trump_suit:
                trump_count += 1
            if self._get_card_points(card) >= 10:
                high_card_count += 1
            hand_points += self._get_card_points(card)

        metrics: Dict[str, int] = {
            'trump_cards': trump_count,
            'high_cards': high_card_count,
            'total_points': score.direct_points + score.pending_points,
            'hand_points': hand_points
        }
        return metrics

    def get_move(self, perspective: PlayerPerspective, leader_move: Optional[Move])-> Move:
        """
        Determine and return the best move based on game phase and current state.

        Args:
            perspective (PlayerPerspective): Current game state and player's view.
            leader_move (Optional[Move]): Move made by leading player if following, None if leading.

        Returns:
            Move: Selected move to play.
        """
        valid_moves: List[Move] = perspective.valid_moves()
        trump_suit: Suit = perspective.get_trump_suit()

        # Phase 1 strategy - Focus on marriages and trump exchanges.
        if perspective.get_phase().value == 1:
            # Check for marriage opportunities, prioritizing trump marriages.
            marriage_moves: List[Move] = [marriage_move for marriage_move in valid_moves if marriage_move.is_marriage()]
            if marriage_moves:
                trump_marriages: List[Move] = [trump_marriage for trump_marriage in marriage_moves if trump_marriage.queen_card.suit == trump_suit]
                return self.__random.choice(trump_marriages) if trump_marriages else self.__random.choice(marriage_moves)

            # Take trump exchange if available.
            exchange_moves: List[Move] = [exchange for exchange in valid_moves if exchange.is_trump_exchange()]
            if exchange_moves:
                return exchange_moves[0]

            return self.__random.choice(valid_moves)

        # Phase 2 strategy - Focus on point maximization.
        metrics: Dict[str, int] = self._evaluate_phase2_position(perspective)
```

```python
        # Leading player strategy.
        if leader_move is None:
            # If holding high cards, try to play them.
            if metrics['high_cards'] > 0:
                regular_moves: List[Move] = []
                for potential_move in valid_moves:
                    if not potential_move.is_marriage() and not potential_move.is_trump_exchange():
                        regular_moves.append(potential_move)

                if regular_moves:
                    # Calculate points for each move.
                    move_points: List[Tuple[Move, int]] = []
                    for current_move in regular_moves:
                        card = self._get_move_card(current_move)
                        if card:
                            points = self._get_card_points(card)
                            move_points.append((current_move, points))

                    # Select from moves worth 10+ points.
                    best_moves: List[Move] = [move for move, points in move_points if points >= 10]
                    if best_moves:
                        return self.__random.choice(best_moves)

            # Try to play lowest non-trump card.
            non_trump_moves: List[Move] = []
            for current_move in valid_moves:
                if (not current_move.is_marriage() and not current_move.is_trump_exchange() and
                    self._get_move_card(current_move) and self._get_move_card(current_move).suit != trump_suit):
                    non_trump_moves.append(current_move)

            if non_trump_moves:
                return min(non_trump_moves, key=lambda move: self._get_card_points(self._get_move_card(move)))

        # Following player strategy.
        else:
            leader_card: Optional[Card] = self._get_move_card(leader_move)
            if not leader_card:
                return self.__random.choice(valid_moves)

            # Try to follow suit.
            following_moves: List[Move] = []
            for follow_move in valid_moves:
                move_card = self._get_move_card(follow_move)
                if (not follow_move.is_marriage() and not follow_move.is_trump_exchange() and
                    move_card and move_card.suit == leader_card.suit):
                    following_moves.append(follow_move)

```

```python
            if following_moves:
                # For valuable tricks, try to win with lowest winning card.
                if self._get_card_points(leader_card) >= 10:
                    winning_moves: List[Move] = []
                    for current_move in following_moves:
                        move_card = self._get_move_card(current_move)
                        if move_card and self._get_card_points(move_card) > self._get_card_points(leader_card):
                            winning_moves.append(current_move)

                    if winning_moves:
                        return min(winning_moves, key=lambda move: self._get_card_points(self._get_move_card(move)))

                    # Play lowest following card if can't/shouldn't win.
                    return min(following_moves, key=lambda move: self._get_card_points(self._get_move_card(move)))

                # For valuable tricks, consider trumping.
                if self._get_card_points(leader_card) >= 10:
                    trump_moves: List[Move] = []
                    for trump_move in valid_moves:
                        move_card = self._get_move_card(trump_move)
                        if (not trump_move.is_marriage() and not trump_move.is_trump_exchange() and
                            move_card and move_card.suit == trump_suit):
                            trump_moves.append(trump_move)

                    if trump_moves:
                        return min(trump_moves, key=lambda move: self._get_card_points(self._get_move_card(move)))

            # Default to random move if no strategy applies.
            return self.__random.choice(valid_moves)

    def create_training_data(num_games: int = 10000)-> List[MLDataBot]:
        """Generate training data by playing games between different bot types."""
        # Clean up and create necessary directories.
        for directory in ['ML_replay_memories', 'ML_models']:
            path: pathlib.Path = pathlib.Path(directory)
            if path.exists():
                shutil.rmtree(path)
            path.mkdir(parents=True, exist_ok=True)

        # Initialize different bot types for diverse training.
        phase2_bot: Phase2FocusedBot = Phase2FocusedBot(random.Random(42))
        rdeep_bot: RdeepBot = RdeepBot(num_samples=12, depth=4, rand=random.Random(43))
        bully_bot: BullyBot = BullyBot(random.Random(44))
```

```
    # Create data collection bots.
    data_bots: List[MLDataBot] = [
        MLDataBot(bot=phase2_bot, replay_memory_location=pathlib.Path('ML_replay_memories/training.txt')),
        MLDataBot(bot=rdeep_bot, replay_memory_location=pathlib.Path('ML_replay_memories/training.txt')),
        MLDataBot(bot=bully_bot, replay_memory_location=pathlib.Path('ML_replay_memories/training.txt'))
    ]

    engine: SchnapsenGamePlayEngine = SchnapsenGamePlayEngine()
    print(f"Playing {num_games} games to collect training data...")

    # Play games to collect training data.
    for game_index in range(num_games):
        if game_index % 100 == 0:
            print(f"Playing game {game_index}/{num_games}")

        selected_bots: Tuple[MLDataBot, MLDataBot] = random.sample(data_bots, 2)
        try:
            engine.play_game(selected_bots[0], selected_bots[1], random.Random(game_index))
        except Exception as error:
            print(f"Error in game {game_index}: {error}")
            continue

    return data_bots

def train_models()-> None:
    """Train both neural network and logistic regression models."""
    try:
        print("Training Neural Network...")
        train_ML_model(
            replay_memory_location=pathlib.Path('ML_replay_memories/training.txt'),
            model_location=pathlib.Path('ML_models/neural_net_model'),
            model_class='NN'
        )

        print("\nTraining Logistic Regression...")
        train_ML_model(
            replay_memory_location=pathlib.Path('ML_replay_memories/training.txt'),
            model_location=pathlib.Path('ML_models/logistic_reg_model'),
            model_class='LR'
        )

    except Exception as error:
        print(f"Error during training: {error}")
```

```python
def play_test_games(bot1: Bot, bot2: Bot, num_games: int = 100)-> Dict[str, float]:
    """Play test games and collect performance statistics."""
    engine: SchnapsenGamePlayEngine = SchnapsenGamePlayEngine()
    stats: Dict[str, float] = {
        'wins': 0,
        'total_time': 0,
        'phase_one_wins': 0,
        'phase_two_wins': 0,
        'total_points': 0,
        'games_played': 0
    }

    # Play test games and collect statistics.
    for game_index in range(num_games):
        try:
            start_time: float = time.time()
            winner, points, score = engine.play_game(bot1, bot2, random.Random(game_index))

            stats['games_played'] += 1
            stats['total_time'] += time.time() - start_time

            # Track win statistics.
            if isinstance(winner, type(bot1)) and id(winner) == id(bot1):
                stats['wins'] += 1
                stats['total_points'] += score.direct_points
                if score.direct_points < 66:
                    stats['phase_one_wins'] += 1
                else:
                    stats['phase_two_wins'] += 1

        except Exception as error:
            print(f"Error in game {game_index}: {error}")
            continue

    # Calculate performance metrics.
    if stats['games_played'] > 0:
        stats.update({
            'win_rate': (stats['wins'] / stats['games_played']) * 100,
            'avg_time': stats['total_time'] / stats['games_played'],
            'avg_points': stats['total_points'] / max(stats['wins'], 1),
            'phase_one_win_rate': (stats['phase_one_wins'] / stats['games_played']) * 100,
            'phase_two_win_rate': (stats['phase_two_wins'] / stats['games_played']) * 100
        })

    return stats
```

```python
def test_models(num_games: int = 200)-> Dict[str, Dict[str, float]]:
    """Test both models against various opponents and each other."""
    try:
        # Initialize models and opponents.
        nn_bot: MLPlayingBot = MLPlayingBot(model_location=pathlib.Path('ML_models/neural_net_model'))
        lr_bot: MLPlayingBot = MLPlayingBot(model_location=pathlib.Path('ML_models/logistic_reg_model'))
        phase2_bot: Phase2FocusedBot = Phase2FocusedBot(random.Random(42))

        opponents: Dict[str, Bot] = {
            'Random': RandBot(random.Random(43)),
            'Bully': BullyBot(random.Random(44)),
            'Phase2': phase2_bot,
            'Rdeep-Basic': RdeepBot(num_samples=8, depth=3, rand=random.Random(45)),
            'Rdeep-Advanced': RdeepBot(num_samples=12, depth=4, rand=random.Random(46))
        }

        results: Dict[str, Dict[str, float]] = {}

        # Test both ML models against all opponents.
        for model_name, model_bot in [('Neural Network', nn_bot), ('Logistic Regression', lr_bot)]:
            print(f"\nTesting {model_name}...")

            for opp_name, opponent in opponents.items():
                print(f"\nPlaying against {opp_name}")

                # Test from both positions.
                print("Testing as first player...")
                stats_first: Dict[str, float] = play_test_games(model_bot, opponent, num_games // 2)
                print("Testing as second player...")
                stats_second: Dict[str, float] = play_test_games(opponent, model_bot, num_games // 2)

                # Combine statistics from both positions.
                total_wins: int = int(stats_first['wins'] + (num_games // 2 - stats_second['wins']))
                phase_one_wins: int = int(stats_first['phase_one_wins'] + stats_second['phase_one_wins'])
                phase_two_wins: int = int(stats_first['phase_two_wins'] + stats_second['phase_two_wins'])

                combined_stats: Dict[str, float] = {
                    'wins': float(total_wins),
                    'games_played': float(num_games),
                    'phase_one_wins': float(phase_one_wins),
                    'phase_two_wins': float(phase_two_wins),
                    'win_rate': (total_wins / num_games) * 100,
                    'phase_one_win_rate': (phase_one_wins / num_games) * 100,
                    'phase_two_win_rate': (phase_two_wins / num_games) * 100
                }

                results[f'{model_name} vs {opp_name}'] = combined_stats

```

```
333
334         # Head-to-head model comparison.
335         print("\nTesting Models Head-to-Head")
336         head_to_head_stats: Dict[str, float] = play_test_games(nn_bot, lr_bot, num_games)
337         results['Model Competition'] = head_to_head_stats
338
339         return results
340
341     except Exception as error:
342         print(f"Error during testing: {error}")
343         traceback.print_exc()
344         return {}
345
346 if __name__ == "__main__":
347     # Execute training and testing pipeline.
348     print("Step 1: Creating training data...")
349     data_bots: List[MLDataBot] = create_training_data(num_games=10000)
350
351     print("\nStep 2: Training models...")
352     train_models()
353
354     print("\nStep 3: Testing models...")
355     results: Dict[str, Dict[str, float]] = test_models(num_games=200)
356
357     # Display final test results.
358     print("\nFinal Results:")
359     print("-" * 50)
360     for match, stats in results.items():
361         print(f"{match}:")
362         print(f"  Wins: {stats['wins']}/{stats['games_played']} ({stats['win_rate']:.1f}%)")
363         print(f"  Phase 1 wins: {stats['phase_one_wins']} ({stats['phase_one_win_rate']:.1f}%)")
364         print(f"  Phase 2 wins: {stats['phase_two_wins']} ({stats['phase_two_win_rate']:.1f}%)")
365         print("-" * 50)
```